

Crivo de Eratóstenes em GPU

1 Definição do Problema

O algoritmo de Crivo de Eratóstenes é utilizado para encontrar todos os números primos até um determinado limite. Para cada primo, é marcado como composto todos os seus múltiplos.

2 Resolução do Problema (PCAM)

2.a Particionamento

Uma descrição básica do algoritmo de Crivo de Eratóstenes para um limite N é a seguinte:

1. Encontre todos os números primos até $\sqrt{N}$ usando o Crivo de Eratóstenes.
2. Marque os múltiplos dos primos encontrados no passo 1 como compostos em um buffer
3. Os números que não forem marcados como compostos em cada segmento são primos.

O objetivo deste projeto é implementar o Crivo de Eratóstenes utilizando a GPU para acelerar o processo de marcação dos múltiplos dos primos, e mostrar a escalabilidade do algoritmo em comparação com a implementação em CPU. O foco deste trabalho são as etapas 2, 3 do algoritmo, que serão paralelizadas na GPU. A etapa 1 será implementada na CPU, pois o número de primos até $\sqrt{N}$ é relativamente pequeno e não justifica a sobrecarga de paralelização na GPU.

A partição do trabalho será: dividir o intervalo de 1 até N em segmentos $[L, R]$ de tamanho igual, cada segmento dividirá o trabalho de marcar os múltiplos dos primos encontrados no passo 1 que se encontram dentro de seu segmento.

2.b Comunicação

2.b.a Buffer de Primos

Para acessar os primos encontrados no passo 1, será usado um buffer na memória global da GPU (aqui poderia ser usado memória constante, porém para N muito grande, o tamanho do buffer pode exceder o limite de memória constante, então optamos por usar a memória global).

2.b.b Buffer de Marcação

Cada bloco de threads terá um buffer local (na memória compartilhada) para marcar os múltiplos dos primos dentro do segmento que estão processando.

O primeiro passo de cada bloco será inicializar esse buffer com valor 1 em todas as posições, indicando que todos os números são inicialmente considerados primos. Em seguida, o bloco deverá esperar as threads terminarem de inicializar o buffer antes de começar a marcar os múltiplos dos primos.

O próximo passo é marcar os múltiplos dos primos encontrados no passo anterior. Cada thread dentro do bloco será responsável por marcar os múltiplos de um subconjunto dos primos (alterando o valor do buffer para 0 nas posições correspondentes aos múltiplos dos primos), como a condição de corrida que acontece nessa etapa é irrelevante (ou seja, se duas threads marcarem o mesmo número como composto, isso não causará um erro lógico), não é necessário usar mecanismos de sincronização para evitar condições de corrida. Em seguida, o bloco deverá esperar as threads terminarem de marcar os múltiplos dos primos antes de contar quantos números primos existem no segmento.

2.b.c Variavel de Resposta

O ultimo passo é contar quantos números primos existem no segmento. Cada thread dentro do bloco será responsável por contar um subconjunto dos números no buffer de marcação, e o resultado será acumulado usando uma variável de redução (utilizando `atomicAdd` para comunicar o resultado entre os blocos de threads). O resultado final será o número total de primos encontrados até N .

2.c Aglomeração

A aglomeração do algoritmo é feita na etapa de marcação dos múltiplos dos primos, ao invés de cada thread ser responsável por marcar todos os múltiplos de um primos específico, cada thread é responsável por marcar um subconjunto dos múltiplos de um conjunto de primos.

Também existe aglomeração de trabalho na variável de resposta, onde cada thread é responsável por contar um subconjunto do buffer de marcação, e apenas depois de contar o número de primos em seu subconjunto, o resultado é acumulado usando uma variável de redução atômica.

2.d Mapeamento

O mapeamento do algoritmo para a GPU é feito da seguinte forma:

- Cada bloco de threads é responsável por processar um segmento $[L, R]$ do intervalo de 1 até N .
- Cada thread dentro do bloco é responsável por inicializar um subconjunto do buffer de marcação para o segmento $[L, R]$.
- Cada thread dentro do bloco é responsável por marcar um subconjunto dos múltiplos dos primos encontrados no passo 1 que se encontram dentro do segmento $[L, R]$.
- Cada thread dentro do bloco é responsável por contar um subconjunto dos números no buffer de marcação para determinar quantos números primos existem no segmento $[L, R]$.